Proposal for: Utilizing Reinforcement Learning with Path Planning Algorithms to Create More Efficient Paths of Travel

Andrew Meyer
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
ameyer2@wpi.edu

Everett Wenzlaff
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
ecwenzlaff@wpi.edu

Sushant Raj
Robotics Engineering
Worcester Polytechnic Institute
Worcester, Massachusetts
skraj@wpi.edu

***Abstract*—In this report, it is proposed to utilize a reinforcement learning program in conjunction with one or more basic path planning algorithms to efficiently optimize a simulated TurtleBot travel path in a 2D environment based on the manipulation of several key parameters with the performance measured against path length, smoothness, travel time, velocity, and clearance.**

I. Introduction – Motivation for Project

In the industrial environment of AMRs, path planners are, in general, deployed en-mass and are not tailored to the customers needs or the warehouse environment in which they are deployed. This results in good, but not excellent performance, and while easier to maintain on the backend, can limit customer output on the front end. These general deployments often cast a wide net and aim to operate on the 'safe side', whether that be wide, nonadjustable clearance, paths around high traffic areas, or creating lanes that avoid foot traffic altogether. In reality, each customer has different needs, and outside of required safety standards, may have certain workplace standards that could offer better performance if utilized properly to tune the AMR deployment. It is obvious that hand tuning any number of parameters for each individual deployment site is unreasonable and the time spent on it would far outweigh any long-term benefits.

II. Project Objectives

To reduce the amount of work input needed to tune path planner parameters, it is proposed that a reinforcement learning (RL) algorithm is deployed for each site and the parameters are adjusted with direct input of the customer to optimize the AMR deployment for their specific site while prioritizing their specific needs.

III. Related Work

To begin, research was conducted to establish any relatable prior work; keywords under study included: path / trajectory planner, reinforcement learning, ROS2, and MPPI. Below are some of the most recent or relevant papers uncovered that either analyzed a similar problem, or utilized part of the proposed approach in a different environment.

A. Comparative Analysis of Local Trajectory Planning Algorithms in ROS2 [1]

In this paper, Elbouhy et. al designed a study that consisted of both a simulation and real world test of a small mobile robot utilizing ROS2 and three different path planners. The path planners were tested in both environments with static and dynamic obstacles and the results were then evaluated to compare algorithms. The study analyzed a DWB (Modified Dynamic Window Approach), MPPI (Model Predictive Path Integral), and RPP (Improved Pure Pursuit) planner and used the following criteria to determine path quality: time, distance, smoothness, minimum clearance, runs completed, and number of recoveries (if obstacle contact is made). After running 10 runs of each planner in both the simulation and real environments, it was clear that MPPI was the fastest but least smooth, DWB operated with the least clearance, and RPP took the most time and traveled the longest distance.

As this relates to this project, this clearly demonstrates that, as expected in some cases, different planners have different strengths and weaknesses. One of the potential goals for this project is to utilize multiple path planners in concert with the RL algorithm and the research conducted in this paper shows that an approach like that could, in some circumstances, optimize a path. It also shows that if certain criteria are considered higher value, then the RL could favor those. For example, if the environment is close quarters and completely static, DWB could be favored for it's low clearance and shorter distance traveled.

B. Local Path Planning Algorithm of Mobile Robot by Modified Optimized Parameters Method [2]

In this paper, Li et. al studied the Dynamic Window Approach (DWA) algorithm and adjusted specific evaluation parameters in an effort to optimize the path planning sequence. DWA is a local path planner that, in this case, works in concert with a global planner. DWA measures and simulates velocity values for the robot over a specific period of time, then with that, it generates and evaluates the paths created and determines the optimal choice. The evaluation function

utilized in this paper looked at the heading, or the difference in angle between the current and goal state, the distance between the trajectory and nearest obstacle, and the speed of the robot, along with weighted modifiers of each (α, β and γ). A simple environment was set up in both a simulation and real-world and the tuning modifiers were adjusted as follows for 3 separate preliminary trials: (0.05, 0.10, 0.10), (0.10, 0.20, 0.20), (0.15, 0.30, 0.30). And it was determined that α had a minimal effect on optimizing the path and was set to 0.10 for the rest of the experiment. β and γ were then tuned between 0.05 and 0.4 in a series of 64 combinations to optimize both at β = 0.04 and γ = 0.36.

In the context of this project, the interest lies in the tuning equations itself. With the goal of this project being to implement a RL-supported algorithm for the path planning, the tuning equation above provides not only quality reference for this project, but also acts as a confident proof-of-concept in that it is known that path planning algorithms can be tuned and optimized on a limited number of parameters and the results are statistically significant and positive.

C. RL-Driven MPPI: Accelerating Online Control Laws Calculation With Offline Policy [3]

In this paper, Qu et. al. present a method of training RL policy offline to accelerate the convergence for the online control law calculation of MPPI in an unmanned aerial vehicle (UAV) application. The method they proposed involved utilizing the Distributional Soft Actor-Critic (DSAC) algorithm to have an RL agent train a pair of neural networks to model distributions for the control costs and rollout guiding policy used directly within the online MPPI control. So in this way, the RL agent improved MPPI by providing better "first guesses" than standard MPPI. The results of their experiments showed that their method converged significantly faster, produced smoother trajectories, and provided higher success rate in reaching the goal than the standard MPPI control law. Additionally, as part of the experiments in this paper, they also trained a separate RL agent to control the UAV directly ("direct RL control") and compared the results to the RL-driven MPPI approach. While the RL agent performed better than the standard MPPI, the RL-driven MPPI approach still outperformed the direct RL control approach in terms of path smoothness and success rate in reaching the goal.

The guiding principle of this paper translates directly to this project, as RL is being used to enhance the strengths of existing local planners instead of replacing them. In fact, results from this paper suggest that the direct RL control approach may not be the most optimal approach for a given path planning application. By leveraging the strengths of both RL and traditional path planning algorithms, this project aims to create a more efficient and adaptable path planning solution for mobile robots in complex environments.

D. Accelerating Deep Reinforcement Learning for Mobile Robot Path Planning with Curriculum Learning and Reward Shaping [4]

In this research paper, Ting et. al. present a modified Proximal Policy Optimization (PPO) in conjunction with a deep reinforcement learning (DRL) algorithm in order to reduce the time needed to move an agent through an environment. Traditional DRL can require millions of episodes in order to converge on a single action. To reduce the number of episodes required and thus the calculation time for each move, they proposed a distance-based weighted dense reward function. The reward function they proposed is seen below where R_g is the reward when the goal is reached, R_c is the collision penalty, R_s is a small step penalty, and R_sh is a distance shaping reward.

$$R = R_g + R_c + R_s + R_sh$$

The critical part of the equation is R_sh, which utilized the euclidean distance to the goal to provide a higher value reward for better steps. For the RL algorithm we are implementing, we intend to use a similar reward function to reduce the learning time needed for each run or environment we simulate.

E. A Hybrid Approach to Indoor Social Navigation: Integrating Reactive Local Planning and Proactive Global Planning [9]

In this paper, Debnath et al. propose a modular navigation framework for indoor social navigation that combines a classical global planner with a deep reinforcement learning (DRL) based local planner. The global planner generates waypoints and assigns Gaussian soft costs around detected pedestrians to anticipate their future positions, while the trained local planner dynamically follows these waypoints while avoiding imminent collisions in real time. The reward function used to train the local planner includes eight terms covering goal reaching, collision penalties, waypoint tracking progress, pedestrian proximity avoidance, and a small per-timestep penalty to encourage efficiency. Through experiments across both cooperative and uncooperative pedestrian scenarios, their hybrid system outperformed both a traditional DWA planner and a standalone RL baseline, with the RL-augmented system achieving a notably lower collision rate.

This paper is directly relevant to the dynamic parameter tuning approach adopted in this project. Their reactive local RL planner continuously adjusts navigation behavior based on local observations (including proximity to obstacles, heading toward waypoints, and distance to goal), at every timestep during navigation. This is very similar to what the proposed implementation in this project is, where the RL agent observes local state and adjusts planner parameters throughout the navigation process rather than making a single configuration decision upfront. Their reward structure combines dense per-step terms with sparse goal and collision signals and their result that a learning-based local planner substantially outperformed fixed-parameter DWA confirms the premise of POT-r: that static, hand-tuned planner parameters can miss out on

meaningful performance beneifts. By leveraging an RL agent to dynamically tune these parameters, this project aims to achieve similar performance gains in path quality and safety across a range of warehouse environments.

F. Deep Reinforcement Learning with PPO for Autonomous Mobile Robot Navigation Using ROS 2 Framework [10]

In this paper, Rana and Kaveendran implement a PPO-based deep reinforcement learning agent to train a TurtleBot3 navigation policy from scratch within a ROS2 environment. The agent observes the robot's surroundings through a LiDAR scan vector and directly outputs normalized linear and angular velocity commands at each timestep, making it the sole navigation controller rather than an augmentation of an existing planner. Training was conducted using a minimal two-term reward function, including a small positive reward for forward progress and a large negative penalty for collisions. This took roughly 4 hours using Stable-Baselines3's PPO implementation integrated with a custom ROS2 Gymnasium environment. The trained policy resulted in better performance, and behavior improvements such as slowing near tight spaces.

This paper is particularly relevant as a validation of a similar implementation being used in this project: ROS2, Gymnasium, PPO, and a TurtleBot platform. The key distinction is that their RL agent replaces the planner entirely by outputting direct velocity commands, whereas this project's agent acts as a meta-controller. Their training time was approximately four hours for a significantly harder problem (the agent must learn full navigation from raw sensor data), providing reassurance that the team's proposed approach is viable.

G. Other Noteworthy Resources

In addition to the papers described above, there were a number of other papers that were reviewed which provided supplemental information for shaping the project. [5] and [6] provided metrics and insights on planner path evaluation which we plan to utilize for our own evaluation metrics. [7] and [8] demonstrated more examples which utilized RL within some sort of path planning framework for a mobile robot (including both direct and indirect RL control).

H. Gaps in Literature

While there are several papers which demonstrate the use of RL in path planning, no papers were identified which utilized RL to configure the parameters for a given local path planning algorithm. This is a critical gap in the literature, as it is well established that the performance of local path planners can be highly sensitive to their parameter settings, and that optimal settings can vary significantly across different environments and use cases. Meanwhile, some of the examples utilizing direct RL control are not very extensible as these methods tend to only train the RL agent to operate in a specific environment; and re-training for more complex environments can be very costly. By leveraging RL to automatically tune these parameters, our project aims to fill this gap and demonstrate a novel approach to optimizing path planning performance in a way that is adaptable to the specific needs of different deployment sites.

IV. Proposed Solution

In order to minimize scope, this project will implement RL within ROS2. Using ROS2 prevents needing to develop a simulation environment and navigation stack from scratch; which allows for more dedicated effort to developing the RL infrastructure and evaluating performance. Additionally, the simulation environment will be constrained to a 2D occupancy grids with a TurtleBot robot model in order to simplify the state space and holonomic constraints for the path planners and RL infrastructure.

A. Performance Evaluation

In order to assess the impacts of incorporating RL within the local path planning process, a set of path quality metrics will be used to evaluate the performance of the RL-augmented planner against a baseline planner with manually tuned parameters across a variety of scenarios (e.g. different combinations of maps, start poses and goal poses). The path quality evaluation metrics will be based on the following criteria:

- Safety (clearance from obstacles)
- Accuracy (success rate in reaching the goal, amount of deviation from global path plan, etc.)
- Efficiency (total time and distance to goal)
- Smoothness (heading angle variance, velocity variance, etc.)

B. RL Architecture

The goal of the RL component of the project is to have a trainable RL agent that can tune planner parameters directly through the Nav2 stack. Independent from the specific RL algorithm (e.g. PPO, DSAC, etc.) to use, two design options were explored for RL architecture of the states, actions, and rewards. It's worth noting that states, actions, and rewards for each design are not fully fleshed out due to the iterative nature of RL development and the tinkering that's often involved within machine learning workflows; but the general structure of each design is described below along with the pros and cons of each approach.

1) One-Time Parameter Tuning during Initialization: With this approach, the RL agent is only responsible for configuring the local planner's parameters one time before the robot begins navigating (e.g. at the start of each episode):

- State would need to include 2D map, start pose, goal pose, and potentially the path generated from the global planner.
- The action space would include presets for the local planner parameters.
- The reward function would be derived from the path quality metrics defined above, with the goal of maximizing path efficiency and smoothness while minimizing travel time and clearance.

This approach composes of one state, one action, and one reward for each episode; so it's relatively straightforward to implement. However, since the one state will need to include the 2D map, the dimensionality of the state space may be very high (e.g. needing to include dimensions for coordinates of all obstacles in the map or pixel and color channel matrices for a pixel based representation of the map). To add to the high dimensionality, since there is only one state/action/reward per episode, the RL agent may require an extremely large number of episodes to learn anything meaningful due to the limited amount of feedback it receives on the quality of its actions per episode.

2) Dynamic Parameter Tuning during Navigation: With this approach, the RL agent would be responsible for dynamically tuning the local planner's parameters throughout the navigation process:

- State would need to include the robot's current pose, velocity, distance from the goal, and information derived from any on-board sensor data (e.g. downsampled LiDAR returns or costmap). The state vector could also be augmented to include prior poses or future waypoints from the global planner to provide more context for decision making.
- Actions could either include presets for the local planner parameters or adjustments to the parameters (e.g. increase clearance by 0.1m, decrease path smoothness by 0.05, etc.).
- The reward function would resemble more standard direct RL control reward functions with a large positive reward for reaching the goal, a large negative reward for collisions, and small negative rewards for all other movements to encourage finding the shortest path to the goal. The reward function could also be expanded to include some of the metrics described above for path quality, such as applying a small negative reward for high path curvature or large changes in velocity to encourage smoother paths.

This is a more "hands on" approach as the RL agent is more involved during the navigation process than the one-time tuning approach; so this approach is likely to be more complex to implement. However, since the state vector is a lower dimensionality and more closely tied to interactions with the environment, the RL agent will have more feedback during each episode to learn from and may converge to an optimal policy more quickly than the one-time tuning approach.

3) Current Decision: Given the time constraints for the project, the dynamic parameter tuning approach will be used going forward. While the one-time tuning approach may seem simpler to implement on the surface due to its relatively "hands off" nature, the lack of feedback combined with potentially high dimensionality of the state space will make RL training too expensive. If anything, the dynamic tuning's "hands on" nature should make it more malleable from an RL training perspective, as it provides the agent the opportunity to learn from each time step during navigation within a single episode. Regardless of the method used to translate a static map into a state vector (as this is a separate complicated problem in and of itself), the state vector of the dynamic tuning approach is much more similar to the inputs of the local planner than the state vector of the one-time tuning approach. This should help facilitate easier learning and decision making for the RL agent (in terms of function approximation) since there's already a strongly established correlation between parameters in the state vector and the performance of the local planner that the RL agent is trying to optimize. Additionally, the actions for the dynamic tuning approach may be more "forgiving" than the action for the one-time tuning approach, which should lead to more stable results. For example, with the one-time tuning approach, the robot is stuck with the same parameters throughout navigation until it reaches the goal or collides with an obstacle; whereas the dynamic tuning approach provides some opportunity for the RL agent to walk back any potential missteps or overcorrections that it makes to the planner parameters before it's too late. Of course, being stuck with the same parameters for the entire episode with the one-time tuning approach isn't any different than running with an RL free planning solution that's been manually tuned; but that just means the available actions (combinations of parameter configurations) for the one-time tuning approach need to be just as carefully curated as the manually tuned parameters for the RL free solution. In other words, the one-time tuning approach still requires meticulous parameter tuning while the dynamic tuning approach allows for some self-correction.

In fact, the one-time tuning approach can be boiled down to a single function approximation problem. Instead of making a "decision" about what preset of values work from a discrete set of choices for a given state, it makes more sense to reframe the problem as a regression problem that takes the state vector as an input and outputs an optimal planner parameter vector. Training this sort of function approximation model through more direct deep learning methods would likely be more efficient than training an RL agent to do something similar. Instead of having to learn through experience, a set of runs could be made offline to create a dataset of state vectors, planner parameter vectors, and results (e.g. a trace of the trajectory taken or the path quality metrics). This dataset could then be fed into a supervised learning model to learn mappings between planner parameters and the resulting path quality for a given state. Once the mappings have been established through the trained model, a separate model could learn to generate optimized planner parameters for a given state by feeding its outputs directly into the already trained model to evaluate the resulting path quality (like an autoencoder with a frozen decoder). This would be a much more direct way to learn the optimal planner parameters for a given state without having to go through the trial and error process of RL training. The dynamic tuning approach, on the other hand, is more naturally suited to an RL framework since it involves making sequential decisions over time with feedback from the environment, which is what RL is designed for. If anything, the one-time tuning approach could be a follow-up project implemented on top of the dynamic tuning approach to serve as the initial parameter configuration. This would give the

RL agent a better starting point to work with and potentially improve the final path quality or speed up convergence.

C. Deliverables

By the end of the semester, the team will have a functional program that runs a 2D simulation utilizing a TurtleBot which needs to navigate through a crowded warehouse-like environment utilizing one or multiple path planners in conjunction with an RL algorithm that will automatically tune the path parameters to optimize the path of travel to align with the request. In this simulation, getting from point A to point B the fastest will not be the only measure of success. The RL algorithm should be able to take in parameter adjustments to the desired path, acceptable smoothness, desired clearance, and zone restrictions in order to create an optimal plan for the TurtleBot.

V. Progress so Far

A. Implemented Simulation Environment

The simulation environment was designed to be cross-platform and reproducible, leveraging Docker to encapsulate all dependencies and ensure consistent behavior across the team's Mac, Windows, and Linux development machines. The environment is structured as a two-layer Docker image: a base capstone_ros_layer image that installs all system-level dependencies (ROS2, Nav2, TurtleBot4 description packages, and visualization tools), and a top-level rbe_capstone image that layers in the project's ROS2 workspace. Separating dependency installation from application code allows the more expensive base layer to be rebuilt infrequently while code iteration remains fast.

The simulation runs on ROS2 Jazzy with the CycloneDDS middleware layer for inter-node communication. The robot model used is the TurtleBot4, whose URDF is loaded via `xacro` and published to the `/robot_description` topic by the `robot_state_publisher` node. Navigation is performed in a 2D occupancy grid environment using the Nav2 stack, which includes `nav2_map_server` for loading pre-built maps and `nav2_lifecycle_manager` for managing the lifecycle of navigation nodes. Maps are stored as paired `.pgm` and `.yaml` files and are selectable at launch time, enabling the team to quickly swap between different warehouse environments as the project progresses.

Visualization is handled by RViz2, which renders the robot model, occupancy map, and planned paths. A single parameterized launch file, `view_robot.launch.py`, brings up all required nodes—the robot state publisher, static TF broadcasters, map server, lifecycle manager, and RViz2—making it straightforward to stand up the full environment with a single command. For the purpose of this project, RViz2 is chosen over Gazebo for visualization because it is more lightweight and the scope of this project does not require a full physics simulation. Gazebo will be consistently slower in launching and running episodes, which is not ideal for iterative tests and tuning of the RL algorithm. RViz2 provides sufficient visualization capabilities for the 2D occupancy grid and planned paths while allowing for faster iteration cycles.

B. Showcase

Figures 1 and 2 below showcase the current state of the simulation environment as described above. Also, progress can be tracked directly through the GitHub respository at https://github.com/SushiRaj99/rbe-capstone.

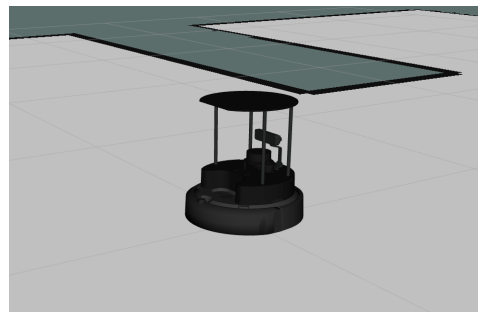
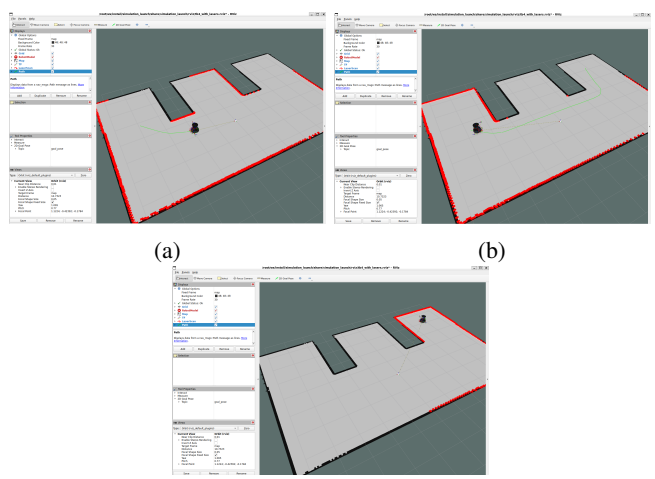


Fig. 1: TurtleBot4 model loaded in RViz2 with a simple occupancy grid map environment for demonstration purposes.



(a) (b) (c)

Fig. 2: TurtleBot4 model scanning and moving through the environment with goal poses set directly through RViz2. In 2a, the robot moves along the generated path (in green) from its initial pose to the first goal pose (in the upper left corner). Then in 2b, a new goal pose was commanded, resulting in a new path (to the upper right corner). Finally, in 2c, the robot reaches the final goal pose. All this navigation was done through manual control; and this sort of process will need to automated in order for the RL agent to be trained. Additionally, as the robot travels from 2a to 2c, it's clear from the LiDAR model scans (in red) that different parts of the environment become occluded. This is an important aspect of the environment that the RL agent will need to learn to account for when tuning the planner parameters.

VI. Next Steps

A. Planned Simulation Development

The next phase of simulation development focuses on transitioning the environment from static visualization to an active, navigable system capable of supporting the reinforcement learning pipeline. This involves three parallel workstreams: navigation integration, trajectory evaluation, and environment expansion.

The immediate priority is wiring in an active odometry source and configuring the Nav2 local planners so the TurtleBot4 can receive and execute navigation goals within the occupancy grid. Both DWB and MPPI will be brought up as configurable Nav2 plugins, with their tunable parameters exposed via the ROS2 parameter server. This is a prerequisite for the RL component, as the agent must be able to set planner parameters and trigger navigation runs programmatically.

Once the robot can navigate, a trajectory evaluation module will be implemented to compute the path quality metrics defined in Section IV. These metrics will ultimately be computed from the `/odom`, `/cmd_vel`, and `/local_costmap` topics published during each navigation run. This module will be implemented as a standalone ROS2 node that subscribes to the relevant topics during an episode and produces a structured result upon goal completion or timeout.

To ensure the RL agent generalizes across site layouts rather than overfitting to a single map, multiple warehouse-style occupancy grid maps will be introduced. Maps will vary along key structural dimensions—corridor width, obstacle density, and open floor area—to expose the agent to the range of layouts described in the project motivation. Together, these components will complete the simulation layer and allow the reinforcement learning environment to be built directly on top of it.

B. Planned RL Development

Once the navigation stack and evaluation is fully functional and automated as described above, the RL pipeline will need to be implemented. The current plan is to implement a PPO-based RL agent using Stable-Baselines3 and integrate it with the ROS2 environment through a custom Gymnasium wrapper. Since machine learning workflows typically involve an iterative process with lots of tinkering, the state vector, action space, and reward function will need to be refined; which will require experimenting with the exact topics that the RL framework subscribes and publishes to. Regardless, the RL framework will need to support both "training" and "suspended" components.

The training component will be responsible for running the full training loop, which includes spawning the episodes, stepping through the environment, sampling actions, and updating the policy. This will require implementing the training component as a management script operating outside of ROS2 so that the environment and all associated nodes can be started and terminated cleanly. The final output of the training component should include a loadable model of the trained RL agent for the suspended component to use.

For the suspended component, the saved model will be loaded and continue to interact with the environment; but the RL agent's policy should be frozen so that actions are deterministic for a given state. This will allow for testing and evaluating the performance of a trained policy while preventing further learning from taking place. An important distinction about the suspended component from the training component is that the suspended component is not responsible for spawning environments; which means that the suspended component is not required to operate outside of ROS2 and can be implemented as a ROS2 node. Implementing the suspended component as a ROS2 node will make it easier to integrate with the rest of the navigation stack and evaluate performance against the baseline planner.

C. Project Timeline

To keep the project on track, the team used the biweekly meeting cadence in conjunction with the class schedule to develop the timeline shown below. The team meets once a week via zoom to go over tasks from previous weeks; as well as set new goals for the upcoming week and assign, or reassign, tasks as needed to keep the project moving. The group is also in constant contact via group chat for day-to-day questions and updates.

- February 6th – Full problem definition and solution outline
- February 20th – Literature review complete, report draft started
- February 27th – Basic sim environment framework built
- March 13th – Basic simulation environment working
- March 20th – Finish proposal report and presentation
- March 27th – Navigation working in simulation
- April 3rd – Path Evaluation working in simulation
- April 17th – RL pipeline implemented and training started
- May 1st – Complete report / presentation

The meeting schedule is shown below for the formal biweekly meetings.

- February 2nd and 6th
- March 2nd, 16th, and 30th
- April 13th and 27th

VII. Task Division

While the team is working collaboratively on all aspects of the project, the following task division was established as a guiding principle to ensure accountability and efficient progress:

- Andrew
  - Unified environment setup and tooling
  - Simulation environment development
- Everett
  - Evaluation definitions and analysis
  - RL pipeline design and scripting
- Sushant
  - Navigation and path planning implementation

– Parameter selection and weights for path planning (and RL tuning)

REFERENCES

[1] S. M. Elbouhy, A. Adamanov, P. M. Braun and H. Wilhelm Rose, "Comparative Analysis of Local Trajectory Planning Algorithms in ROS2," 2025 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR), Palermo, Italy, 2025, pp. 1-6, doi: 10.1109/SIMPAR62925.2025.10979015. keywords: Measurement;Navigation;Trajectory planning;Heuristic algorithms;Operating systems;Warehousing;Transportation;Safety;Autonomous robots;Testing;ROS2;Local Planner;DWB;MPPI;RPP,

[2] H. Li, W. Zhang, Z. Dong, J. Yang and J. Lu, "Local Path Planning Algorithm of Mobile Robot by Modified Optimized Parameters Method," 2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), Chongqing, China, 2021, pp. 1014-1018, doi: 10.1109/IAEAC50856.2021.9391102. keywords: Systematics;Simultaneous localization and mapping;Navigation;Heuristic algorithms;Path planning;Mobile robots;Information technology;robot navigation;local path planning;DWA;simulation;orthogonal test,

[3] Y. Qu et al., "RL-Driven MPPI: Accelerating Online Control Laws Calculation With Offline Policy," in IEEE Transactions on Intelligent Vehicles, vol. 9, no. 2, pp. 3605-3616, Feb. 2024, doi: 10.1109/TIV.2023.3348134. keywords: Optimal control;Vehicle dynamics;Trajectory;Complex systems;Real-time systems;Reinforcement learning;Predictive control;Autonomous aerial vehicles;Model predictive control (MPC);reinforcement learning (RL);unmanned aerial vehicle (UAV),

[4] L. Ting, H. B. Y. Ali and M. K. Chamran, "Accelerating Deep Reinforcement Learning for Mobile Robot Path Planning with Curriculum Learning and Reward Shaping," 2025 IEEE 9th International Conference on Software Engineering and Computer Systems (ICSECS), Pekan, Pahang, Malaysia, 2025, pp. 343-347, doi: 10.1109/ICSECS65227.2025.11279207. keywords: Training;Three-dimensional displays;Systematics;Navigation;Deep reinforcement learning;Path planning;Mobile robots;Optimization;Faces;Convergence;mobile robot;path planning;deep reinforcement learning;training efficiency;curriculum learning;reward shaping

[5] F. Pimentel and P. Aquino, "Performance Evaluation of ROS Local Trajectory Planning Algorithms to Social Navigation," Oct. 2019, doi: https://doi.org/10.1109/lars-sbr-wre48964.2019.00035.

[6] Isira Naotunna and Theeraphong Wongratanaphisan, "Comparison of ROS Local Planners with Differential Drive Heavy Robotic System," vol. 4, pp. 1-6, Dec. 2020, doi: https://doi.org/10.1109/icamechs49982.2020.9310123.

[7] C.-C. Wong, K.-D. Weng, and B.-Y. Yu, "Multi-Robot Navigation System Design Based on Proximal Policy Optimization Algorithm," Information, vol. 15, no. 9, p. 518, Aug. 2024, doi: https://doi.org/10.3390/info15090518.

[8] F. Almeida, G. Leão, and A. Sousa, "An Educational Kit for Simulated Robot Learning in ROS 2," Robot 2023: Sixth Iberian Robotics Conference, vol. 2, pp. 513-525, doi: https://doi.org/10.1007/978-3-031-59167-9_4.

[9] A. Debnath, G. J. Stein, and J. Košecká, "A Hybrid Approach to Indoor Social Navigation: Integrating Reactive Local Planning and Proactive Global Planning," *2025 IEEE International Conference on Robotics and Automation (ICRA)*, Atlanta, USA, May 2025, pp. 10432–10438, doi: https://doi.org/10.1109/ICRA55743.2025.11127475.

[10] A. Rana and B. Kaveendran, "Deep Reinforcement Learning with PPO for Autonomous Mobile Robot Navigation Using ROS 2 Framework," *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, vol. 13, no. VII, Jul. 2025, doi: https://doi.org/10.22214/ijraset.2025.73330.